Machine Learning Operations (MLOps) AIMLCZG523

Assignment 01

Roll No – 2025CT05025 | Name – Abhik Sasmal

Use Case : Heart Disease Classification

Repository: https://github.com/abhik-sasmal/mlops-assignment

Architecture overview

UCI dataset → cleaning/EDA → preprocessing and two classifiers → evaluation and MLflow → saved model → FastAPI → Docker → local Kind Kubernetes → health probes and HTTP access logs.

Abstract

This project implements an end-to-end machine-learning workflow for binary classification using the Cleveland subset of the UCI Heart Disease dataset. It includes data acquisition, data-quality checks, exploratory analysis, preprocessing, comparison of two classifiers, cross-validation, MLflow tracking, automated tests, a FastAPI service, Docker packaging, and local Kubernetes deployment using Kind.

The best recorded model was Logistic Regression, with 0.885 test accuracy and 0.966 ROC-AUC on a held-out set of 61 records. These are experimental results on a small dataset and do not establish clinical validity. The API is for education only.

1. Problem and dataset

The objective is to classify a record as no disease recorded (0) or disease recorded (1). The source contains 303 records and 14 columns: 13 input features and the original target `num`. The target was converted using `num == 0 → 0` and `num > 0 → 1`. The resulting target has 164 class-0 records (54.13%) and 139 class-1 records (45.87%).

The raw data has no duplicate rows and six missing feature values: four in `ca` and two in `thal`. The dataset is small and historical, and should not be considered representative of all clinical populations.

2. Data preparation and EDA

Scripts retrieve the UCI dataset, validate expected columns and target values, convert values to numeric types, and treat `?` as missing. Missing feature values are preserved until imputation in the model pipeline. EDA scripts generate target distribution, missing-value summaries, numerical and categorical distributions, and a correlation heatmap.

In the descriptive summaries, mean `thalach` was 158.38 for class 0 and 139.26 for class 1; mean `oldpeak` was 0.59 and 1.57 respectively. These are associations in this dataset, not causal conclusions.

3. Feature engineering and models

The numerical features are `age`, `trestbps`, `chol`, `thalach`, and `oldpeak`; the remaining eight input features are treated as categorical/integer-coded variables. Numerical values are median-imputed and standardized. Categorical values are imputed with the most frequent value and one-hot encoded, with unknown categories ignored. These transformations are inside each scikit-learn pipeline to avoid fitting preprocessing on held-out test records.

Logistic Regression and Random Forest were compared using a stratified 80/20 split with seed 42 (242 training and 61 test records). Five-fold StratifiedKFold cross-validation was run on the training partition. The test partition was reserved for final comparison.

4. Experiment tracking and results

MLflow records model parameters, cross-validation and test metrics, confusion-matrix artifacts, and trained model artifacts. Metrics include accuracy, precision, recall, F1, and ROC-AUC. A JSON metrics summary is written to `reports/model_metrics.json`; fitted pipelines are saved under `artifacts/models/`.

Model	CV acc.	CV F1	CV AUC	Test acc.	Test prec.	Test recall	Test F1	Test AUC
Logistic Regression	0.847	0.825	0.903	0.885	0.839	0.929	0.881	0.966
Random Forest	0.814	0.794	0.896	0.869	0.813	0.929	0.867	0.949

5. Model results

Logistic Regression achieved mean cross-validation accuracy 0.847 (standard deviation 0.010), F1 0.825, and ROC-AUC 0.903. On the held-out test set it achieved accuracy 0.885, precision 0.839, recall 0.929, F1 0.881, and ROC-AUC 0.966. Its confusion matrix was [[28, 5], [2, 26]], with rows representing true classes and columns predicted classes.

Random Forest achieved mean cross-validation accuracy 0.814 (standard deviation 0.031), F1 0.794, and ROC-AUC 0.896. Test accuracy was 0.869, precision 0.813, recall 0.929, F1 0.867, and ROC-AUC 0.949. Its confusion matrix was [[27, 6], [2, 26]]. Logistic Regression was selected because it performed better on the recorded CV and test metrics and is simpler.

6. Packaging, API, tests, and CI

The selected pipeline is serialized as `artifacts/models/logistic_regression.joblib`. FastAPI loads this artifact and exposes `/health` and `/predict`. The prediction response includes the binary class, label, class probabilities, model identifier, and educational-use disclaimer. Request fields are validated by the API schema.

Tests cover data cleaning and target validation, model predictions/probabilities, and API behavior including invalid requests. The last recorded local test run passed 12 tests. GitHub Actions runs the test suite on configured pushes and pull requests. The workflow should only be described as test CI unless linting and model training are explicitly added and verified.

7. Docker and Kubernetes deployment

The Docker image packages the Python 3.9 runtime, pinned dependencies, application source, and saved model. The service listens on port 8000. Local verification returned `{"status":"ok","model_loaded":true}` from `/health`, and `/predict` returned a prediction and probability distribution for the example input.

The app was deployed to a local Kind cluster. The manifest defines one Deployment replica, a NodePort Service, readiness/liveness probes against `/health`, and CPU/memory requests and limits. Rollout reached one ready replica. Kubernetes endpoints were tested using `kubectl port-forward service/heart-disease-api 8001:8000`. This is a local Kubernetes deployment, not public-cloud deployment.

8. Monitoring and operational visibility

Basic operational visibility is provided by FastAPI/Uvicorn HTTP access logs and Kubernetes readiness/liveness probes. Logs showed successful `/health` requests, including probe traffic. These features confirm basic health and record requests but are not a metrics dashboard. A future improvement is to expose request counts, latency, and HTTP error rates and visualize them with Prometheus/Grafana.

9. Reproducibility and architecture

The pipeline is: UCI acquisition → validation/cleaning and EDA → preprocessing and model training → cross-validation and test evaluation → MLflow tracking and saved model → FastAPI → Docker → local Kind Kubernetes. Reusable source modules, scripts, and tests are separated. Dependencies are pinned in `requirements.txt`.

Reproduction commands and operational steps are in the repository README. From a clean setup, install dependencies, run the data scripts and training script, run `PYTHONPATH=src python -m pytest tests/ -v`, build the Docker image, and verify the API. For Kubernetes, create the Kind cluster, load the image, apply the manifest, wait for rollout, and test through port-forwarding.

10. Conclusion

The dataset contains only 303 records and the test partition has 61 records; metrics are uncertain and can vary with the split. The data is not a current, diverse, externally validated clinical cohort. No claim of clinical utility, diagnosis, or generalization to other populations is warranted.